

Generator semnal Pulse-Width Modulation

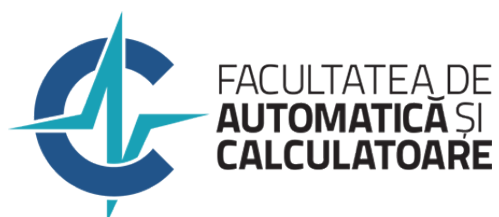
Proiect Arhitectura Calculatoarelor

Proiect realizat de:

Anghel Elena-Cristina

Nedelcu Bianca-Nicoleta

Toea Mariana



Cuprins

1	Introducere	2
2	Arhitectura Modulului	2
3	Detalii de Implementare	3
3.1	SPI Bridge	3
3.1.1	Recepția prin MISO	3
3.1.2	Transmisia pe MOSI	4
3.1.3	Sincronizarea ceasurilor	4
3.2	Decodorul de instrucțiuni	4
3.2.1	Funcționarea AFD-ului	5
3.2.2	Elemente de siguranță și inițializare	5
3.2.3	Observații privind suportul pentru byte high/low	5
3.2.4	Concluzie	6
3.3	Blocul de regiștri	6
3.3.1	Interfața modulului	6
3.3.2	Structura adreselor	7
3.3.3	Logica de scriere (Secvențială)	7
3.3.4	Logica de citire (Combi-națională)	7
3.3.5	Caracteristici importante	8
3.3.6	Concluzie	8
3.4	Implementarea modulului de numărare	8
3.4.1	Organizarea codului și modul de proiectare	8
3.4.2	Sistemul de scalare a timpului (prescaler-ul)	9
3.4.3	Numărătorul principal și gestiunea direcției	9
3.4.4	Strategia de reset și siguranță	9
3.5	Implementarea modulului generator PWM	10
3.5.1	Modul de generare a semnalului	10
3.5.2	Logica combinațională și modurile de aliniere	10
3.5.3	Mecanismul de siguranță și memorarea stării	10
3.5.4	Ieșirea finală și sincronizarea	11
4	Concluzii	11

1 Introducere

Modulația în lățime de impuls (PWM - Pulse Width Modulation) reprezintă o tehnică fundamentală în electronica digitală modernă, fiind omniprezentă în controlul dispozitivelor periferice, de la ajustarea intensității luminoase a elementelor optoelectronice (LED-uri) până la reglarea fină a turației motoarelor electrice. Capacitatea de a simula un semnal analogic folosind mijloace exclusiv digitale face din modulele PWM componente critice în arhitectura microcontrolerelor moderne, precum ATmega328P.

Prezentul document detaliază proiectarea și implementarea unui periferic digital dedicat generării semnalelor PWM, dezvoltat folosind limbajul de descriere hardware Verilog. Obiectivul principal al proiectului este realizarea unui modul robust, modular și complet configurabil, capabil să interfațeze cu un sistem gazdă (master) pentru controlul parametrilor de semnal în timp real.

Arhitectura propusă permite controlul dinamic al caracteristicilor universale ale semnalului PWM:

- **Perioada semnalului:** Timpul total al unui ciclu, definit prin valoarea maximă de numărare.
- **Factorul de umplere (Duty Cycle):** Procentul din perioadă în care semnalul este activ, determinând puterea medie livrată sarcinii.
- **Alinierea:** Modul de generare a impulsului (aliniat la stânga, la dreapta sau nealiniat).

Pentru a asigura flexibilitatea și integrarea facilă în sisteme complexe, perifericul include un divizor de frecvență programabil (*prescaler*) pentru scalarea bazei de timp și comunică cu exteriorul printr-o interfață serială standardizată de tip **SPI Slave** (Serial Peripheral Interface).

În secțiunile următoare, vom analiza structura internă a modului, descompusă în sub-blocuri funcționale (Bridge SPI, Decodor de Instrucțiuni, Bloc de Registri, Numărător și Generator PWM), și vom prezenta particularitățile implementării hardware, evidențiind soluțiile adoptate pentru sincronizare și gestionarea logicii combinaționale și secvențiale.

2 Arhitectura Modulului

Arhitectura perifericului implementat modelează un generator PWM configurabil, similar celor integrate în microcontrolere. Sistemul este construit modular, astfel încât fiecare bloc intern are un rol bine definit, iar împreună permit comunicarea cu exteriorul, programarea configurațiilor și generarea semnalului PWM final. Comunicarea cu utilizatorul se face prin protocolul SPI, prin intermediul unui bridge care transformă fluxul serial de biți în pachete paralele și sincronizate cu ceasul intern al perifericului.

Byte-urile recepționate prin SPI sunt interpretate de un decodor de instrucțiuni, implementat ca o mașină de stări finite, care extrage informațiile necesare (tipul operației, adresa și regiunea de memorie accesată) și generează tranzacții de citire sau scriere către blocul de registre. Acest bloc de registre reprezintă memoria internă a perifericului și reține toate setările ce definesc modul de funcționare al generatorului PWM: perioada, valorile de comparație, prescalarea, direcția contorului și modurile de aliniere.

Valorile configurate sunt apoi utilizate de numărătorul programabil, care generează baza de timp a semnalului PWM, și de generatorul PWM propriu-zis, care produce semnalul final în funcție de comparațiile efectuate între contor și valorile setate. În acest fel, perifericul poate fi controlat din exterior și poate genera un semnal PWM cu parametri complet configurabili prin intermediul interfeței SPI.

3 Detalii de Implementare

3.1 SPI Bridge

Modulul `spi_bridge` reprezintă interfața dintre protocolul SPI al masterului extern și restul perifericului intern. Implementarea este sincronizată cu semnalul de ceas SPI (`sclk`) și respectă specificațiile $CPOL = 0$ și $CPHA = 0$, în care:

- Datele sunt transmise către master pe frontul descendent al lui `sclk` (`data_out` -> MOSI)
- Datele sunt recepționate de la master pe frontul ascendent al lui `sclk` (MiSO -> `data_in`)

Transmiterea și recepționarea datelor se realizează doar atunci când semnalul Chip Select (`cs_n`) este activ pe nivel logic 0, ceea ce indică faptul că o sesiune de comunicare SPI este activă.

Pentru implementarea modulului am declarat următorii parametri interni de tip `reg`:

- `data_shift[7:0]` – acumulează, bit cu bit, datele recepționate de la master prin linia MISO.
- `bit_cnt` – contor pe 3 biți care urmărește câți biți au fost recepționați, determinând momentul în care s-a transmis un byte.
- `byte_done` – flag folosit pentru resetarea internă a contorului după capturarea unui byte complet și pentru sincronizarea cu `clk` pentru a transmite datele către `instr_dcd`.
- `data_final` – reține valoarea finală a byte-ului citit în `data_shift`

Transmiterea datelor se resetează în două cazuri: semnalul `cs_n` devine HIGH sau semnalul `rst_n` devine LOW. Pentru a reține când are loc o întrerupere în transmisie am folosit un `wire shift_reset` care este activ dacă măcar una dintre cele două condiții de reset este îndeplinită.

3.1.1 Recepția prin MISO

Recepția datelor prin MISO se realizează printr-un bloc `always` care reacționează la frontul crescător al lui `sclk` și la frontul crescător al lui `shift_reset`.

În momentul în care `shift_reset` este activ, sunt inițializate valorile pentru `data_shift`, `bit_cnt`.

Când `shift_reset` devine inactiv, începe citirea datelor de pe MISO: la fiecare front crescător al `sclk`, `data_shift` realizează o shiftare la stânga, bitul recepționat prin MISO este plasat pe poziția LSB, iar `bit_cnt` se incrementează.

Capturarea byte-ului final se realizează într-un bloc **always** care reacționează la frontul crescător al `sclk` și frontul descrescător al `rst_n`. În momentul în care `rst_n` este activ, inițializez valorile pentru `byte_done` și `data_final`.

Atunci când `bit_cnt = 7` (adică s-au primit 8 biți), și dacă `cs_n` este încă activ, inversez starea semnalului `byte_done`, indicând terminarea citirii unui byte. Acest byte este pus în `data_final`, capturând și ultimul bit trimis prin MISO.

3.1.2 Transmisia pe MOSI

Am tratat transmisia datelor pe MOSI separat, deoarece aceasta folosește frontul descrescător al semnalului `sclk`. Am folosit de asemenea și un `reg[7:0] data_out_shift` în care am reținut un byte primit de pe MOSI. Atunci când `shift_reset` este activ inițializez `data_out_shift`.

La fiecare front descrescător al ceasului, dacă `bit_cnt = 0` (marchează începutul unui nou byte) datele de transmis din `data_out` sunt încărcate în `data_out_shift`. În continuare, la fiecare front descrescător ulterior (când `bit_cnt != 0`), `data_out_shift` face câte o shiftare la stânga, trimițând bitul current (MSB) pe MOSI și adăugând 0 pe LSB.

3.1.3 Sincronizarea ceasurilor

Transmisia datelor de la `spi_bridge` către `inst_dcd` de realizează atunci când cele două ceasuri `sclk` și `clk` sunt sincronizate. Pentru a realiza acest lucru am utilizat:

- `toggle_sync[1:0]` – adaug mereu valoarea lui `byte_done` care arată dacă s-a terminat citirea unui byte prin inversarea sa.
- `toggle_prev` – în care rețin valoarea anterioară a lui `toggle_sync[1]`.

Sincronizarea se realizează într-un bloc **always** care reacționează la frontul crescător al lui `clk` și frontul descrescător al lui `rst_n`. Când este activ `rst_n` inițializez `toggle_sync`, `toggle_prev`, `byte_sync` și `data_in`.

La fiecare front crescător al ceasului, `toggle_sync` realizează o shiftare la stânga pentru a aduce valoarea anterioară de pe `toggle_sync[0]` pe MSB și îi adaug noua valoare a lui `byte_done` pe LSB. În `toggle_prev` rețin valoarea anterioară de pe MSB a lui `toggle_sync`. Realizez o comparație între cele două (valoarea curentă din MSB-ul lui `toggle_sync` și valoarea de la ciclul anterior reținută în `toggle_prev`):

- dacă sunt diferite înseamnă că starea lui `byte_done` s-a schimbat, deci s-a realizat citirea unui byte. Îl transmit mai departe către `instr_dcd` în `data_in` și setez flag-ul de `byte_sync` ca fiind 1 pentru a marca transmiterea unui byte
- dacă am aceeași valoare `byte_sync` rămâne 0

3.2 Decodorul de instrucțiuni

Modulul `instr_dcd` reprezintă decodorul de instrucțiuni pentru fluxul de date provenit de la interfața SPI. Rolul său este de a interpreta fiecare comandă primită sub forma a două bytes: un byte de setup și un byte de date. În funcție de byte-ul de setup, modulul decide dacă urmează o operație de citire sau scriere asupra unui registru din sistem și generează semnalele corespunzătoare (`read`, `write`, `addr`, `data_write`) pentru blocul de registre.

De asemenea, pentru operațiile de citire, datele preluate din registru sunt transmise în `data_out`.

Modulul implementează un **automat finit determinist (AFD)** cu două stări principale, `STATE_SETUP` și `STATE_DATA`, și folosește exclusiv logică secvențială sincronizată cu semnalul de ceas `clk` și reset asincron activ pe 0 (`rst_n`). Toate semnalele interne și de ieșire sunt resetate la valoarea inițială în momentul activării resetului.

3.2.1 Funcționarea AFD-ului

1. STATE_SETUP În această stare modulul așteaptă semnalul `byte_sync` care indică sosirea byte-ului de setup. Byte-ul de setup conține trei informații principale:

- **Bitul 7:** determină tipul operației (0 pentru citire, 1 pentru scriere).
- **Bitul 6:** determină dacă se scrie/citește byte-ul high sau low (0 pentru high, 1 pentru low).
- **Biti 5:0:** reprezintă adresa de bază a registrului vizat.

Pe baza acestor informații, modulul setează registrele interne `is_write`, `is_high` și `saved_addr`. Adresa efectivă pentru operație este calculată astfel încât byte-ul low să folosească adresa de bază, iar byte-ul high să folosească adresa de bază plus unu. Dacă operația este de citire, semnalul `read` este activat pentru un ciclu de ceas.

După preluarea și decodarea byte-ului de setup, AFD-ul trece în starea `STATE_DATA` pentru procesarea byte-ului de date.

2. STATE_DATA În această stare modulul gestionează byte-ul de date asociat comenzii primite anterior. Dacă operația este de citire, datele citite din registru (`data_read`) sunt transferate către `data_out`. În cazul unei operații de scriere, byte-ul primit în `data_in` este plasat în `data_write`, iar semnalul `write` este activat pentru un singur ciclu de ceas. După procesarea byte-ului de date, AFD-ul revine automat în starea `STATE_SETUP`, pregătit pentru următoarea comandă.

Acest comportament asigură că semnalele `read` și `write` sunt întotdeauna *pulse* de un singur ciclu de ceas, prevenind activarea neintenționată a registrelor.

3.2.2 Elemente de siguranță și inițializare

- În cazul unei stări neprevăzute, AFD-ul revine automat în `STATE_SETUP`.
- La activarea resetului asincron (`rst_n = 0`), toate registrele interne și semnalele de ieșire sunt resetate, inclusiv adresele, semnalele R/W și buffer-ele de date.
- Această inițializare asigură un comportament determinist și previzibil la pornirea modulului.

3.2.3 Observații privind suportul pentru byte high/low

Modulul este conceput pentru a lucra cu registre de 16 biți, dar comunică la nivel de *byte*. Logica de adresare Low/High asigură că fiecărui byte din registru îi corespunde adresa corectă. Această abordare permite integrarea ușoară cu SPI și cu sistemul de registre, respectând protocolul „setup byte → data byte”.

3.2.4 Concluzie

Modulul `instr_dcd` realizează decodarea instrucțiunilor SPI și generarea semnalelor necesare pentru accesul la registre, implementând:

- Interpretarea byte-ului de setup și determinarea tipului de operație și adresei efective.
- Gestionarea corectă a byte-urilor high și low pentru registrele pe 16 biți.
- Generarea corectă a semnalelor `read` și `write` cu durata unui singur ciclu.
- Siguranță la pornire și revenirea automată în starea inițială pentru orice condiție neașteptată.

Prin această arhitectură, modulul poate fi integrat eficient într-un sistem mai larg, comunicând prin SPI și controlând registrele în mod sincron și sigur.

3.3 Blocul de regiștri

Modulul `regs` reprezintă blocul de regiștri al sistemului, responsabil pentru stocarea parametrilor legați de counter și generatorul PWM. Acesta primește comenzi de citire și scriere de la decodorul de instrucțiuni (`instr_dcd`) și gestionează atât registrele interne pe 16 biți, cât și semnalele de control asociate.

3.3.1 Interfața modulului

Intrări principale:

- `clk`, `rst_n` – semnale de ceas și reset asincron activ pe 0.
- `read`, `write` – semnale de control generate de decodorul de instrucțiuni, active doar 1 ciclu de ceas.
- `addr[5:0]` – adresa registrului vizat.
- `data_write[7:0]` – datele care trebuie scrise într-un registru.
- `counter_val[15:0]` – valoarea curentă a counter-ului, utilizată pentru citire.

Ieșiri principale:

- `data_read[7:0]` – byte-ul citit din registru.
- `period[15:0]`, `compare1[15:0]`, `compare2[15:0]` – registrele de tip 16 biți.
- `en`, `count_reset`, `upnotdown`, `pwm_en` – semnale de control pentru counter și generator PWM.
- `prescale[7:0]`, `functions[7:0]` – parametri de configurare suplimentari.

3.3.2 Structura adreselor

Fiecare registru intern ocupă unul sau doi bytes consecutivi, iar adresele au fost definite astfel:

- `ADDR_PERIOD_L/H` – byte-urile low și high ale registrului `period`.
- `ADDR_COMPARE1_L/H`, `ADDR_COMPARE2_L/H` – byte-urile low și high ale registrilor `compare1` și `compare2`.
- `ADDR_COUNTER_EN`, `ADDR_COUNTER_RESET`, `ADDR_UPNOTDOWN`, `ADDR_PWM_EN` – semnale de control de 1 bit.
- `ADDR_COUNTER_VAL_L/H` – byte-urile low și high ale valorii counter-ului.
- `ADDR_PRESCALE`, `ADDR_FUNCTIONS` – registre suplimentare pe 8 biți.

3.3.3 Logica de scriere (Secvențială)

Scrierea în registre se realizează la frontul pozitiv al ceasului sau la resetul asincron. Principalele caracteristici sunt:

- La activarea resetului (`rst_n = 0`), toate registrele sunt inițializate la 0, cu excepția `upnotdown` care este 1.
- Registrul `count_reset` include un mecanism de auto-clear: după 2 cicluri de ceas, semnalul este resetat automat. Aceasta se realizează printr-un counter de 2 biți (`reset_clear_cnt`).
- Operațiile de scriere se efectuează doar dacă semnalul `write` este activ. În funcție de adresă, modulul scrie fie un byte al registrului de 16 biți, fie un semnal de control de 1 bit.

3.3.4 Logica de citire (Combinațională)

Citirea registrului se realizează folosind un bloc combinational `always @(*)`, care returnează byte-ul corespunzător adresei:

- Pentru registrele pe 16 biți, se returnează byte-ul low sau high în funcție de adresă.
- Pentru semnalele de control pe 1 bit, valoarea este extinsă cu 7 biți 0.
- Pentru registrele write-only, precum `count_reset`, citirea returnează 0.
- Pentru adrese neutilizate, se returnează 0, conform specificațiilor.

3.3.5 Caracteristici importante

- Modulul asigură un comportament determinist și sincronizat, prevenind erorile de citire/scriere simultană.
- Semnalele de control (`en`, `pwm_en`, `upnotdown`) sunt configurabile prin scriere directă, dar pot fi citite pentru verificare.
- Separarea logicii secvențiale (scriere) de cea combinațională (citire) asigură claritate și compatibilitate cu decodorul de instrucțiuni.
- Toate operațiile respectă principiul pulsului de un singur ciclu pentru semnalele de control generate (`write` și `read`).

3.3.6 Concluzie

Modulul `regs` furnizează acces controlat și sigur la registrele de sistem, asigurând:

- Stocarea și manipularea parametrilor counter și PWM.
- Gestionarea corectă a registrelor pe 16 biți cu acces la bytes low și high.
- Siguranță la reset și auto-clear pentru semnale puls.
- Compatibilitate completă cu modulul `instr_dcd`.

3.4 Implementarea modului de numărare

În această secțiune, voi detalia modul în care am proiectat și structurat numărătorul, care reprezintă „motorul” întregului periferic PWM. Rolul meu a fost să creez o bază de timp stabilă și flexibilă, capabilă să genereze evenimente la intervale precise de timp, indiferent de cât de rapid este ceasul sistemului.

3.4.1 Organizarea codului și modul de proiectare

Pentru acest modul, am decis să folosesc o structură cât mai ordonată, astfel încât fluxul datelor să fie ușor de urmărit și să nu apară erori de sincronizare. Am decis să separ complet logica în două blocuri distincte:

- **Logica combinațională:** Unde calculez „pe loc” valorile de prag și semnalele de control, precum semnalul `tick` și `prescale_limit`
- **Logica secvențială:** Unde realizez actualizarea registrelor (memoriei) la fiecare front crescător al ceasului (`posedge clk`).

Această separare nu este doar o chestiune de stil: am ales-o pentru că elimină riscul de a genera semnale parazite și face codul mult mai ușor de urmărit și depanat.

3.4.2 Sistemul de scalare a timpului (prescaler-ul)

O problemă majoră pe care am identificat-o a fost frecvența prea mare a ceasului de sistem. Dacă aș număra la fiecare ciclu de ceas, numărătorul pe 16 biți ar da *overflow* mult prea repede pentru a fi util în aplicații reale.

Pentru a rezolva asta, am implementat un **prescaler**. Acesta funcționează ca un divizor de frecvență programabil prin registrul **PRESCALE**.

- **Calculul limitei:** În partea combinațională, am folosit următoarea logică pentru a determina pragul de divizare:

```
assign prescale_limit = (prescale < 8'd16) ?  
                        ((16'd1 << prescale) - 16'd1) : 16'd  
                        hFFFF;
```

Am implementat acest lucru folosind operația de shiftare la stânga ($1 \ll \text{prescale}$), deoarece este mult mai eficientă în hardware decât o funcție exponențială clasică.

- **Hack de siguranță:** Am adăugat o verificare pentru a limita **prescale** la maximum 15. Dacă utilizatorul introduce o valoare mai mare, forțez limita la **16'hFFFF** pentru a preveni un overflow de shiftare care ar putea bloca sistemul.
- **Generarea tick-ului:** Semnalul **tick** este cel care dă „permisiunea” numărătorului principal să facă un pas:

```
assign tick = (prescale_cnt >= prescale_limit) && en;
```

Acesta devine activ doar când contorul intern al prescaler-ului a atins limita și modulul este activat (**en == 1**).

3.4.3 Numărătorul principal și gestiunea direcției

Inima modulului este registrul **count_val** pe 16 biți, care oferă o rezoluție de $2^{16} = 65536$ de pași. Am implementat două moduri de funcționare, controlate prin bitul **upnotdown**:

1. **Modul incrementare (Up):** Număr de la 0 până la valoarea setată în registrul **PERIOD**. Când **count_val >= period**, acesta revine la 0 (alinieare la stânga).
2. **Modul decrementare (Down):** Încep de la **PERIOD** și cobor până la 0. La atingerea valorii minime, reîncarc automat valoarea din **PERIOD** (alinieare la dreapta).

3.4.4 Strategia de reset și siguranță

Am acordat o atenție deosebită modulului în care sistemul revine la o stare inițială prin două tipuri de reset:

- **Reset asincron (rst_n):** Acționează imediat, independent de ceas, punând totul pe 0.
- **Reset sincron (count_reset):** Declanșat prin software (adresa 0x07). Am programat sistemul să fie „inteligent”:

```
else if (count_reset) begin  
    count_val <= (upnotdown) ? 16'd0 : period;  
end
```

Dacă suntem în modul *Down*, resetarea mă duce în **period**, deoarece acesta este punctul de start corect pentru numărarea inversă.

De asemenea, dacă **en** este dezactivat, contorul prescaler-ului se resetează la 0 pentru a asigura un start curat la următoarea activare.

3.5 Implementarea modului generator PWM

După ce am pus la punct baza de timp prin intermediul numărătorului, pasul următor a fost crearea unui modul capabil să „traducă” acea numărătoare într-un semnal fizic utilizabil. Generatorul PWM reprezintă interfața directă cu exteriorul, fiind cel care comandă dispozitivele externe, precum un LED, un motor sau un driver de tranzistori.

3.5.1 Modul de generare a semnalului

În proiectarea acestui modul, am urmărit să ofer utilizatorului o flexibilitate cât mai mare. Caracteristicile semnalului sunt dictate de valorile încărcate în regiștrii **PERIOD**, **COMPARE1** și **COMPARE2**, dar și de configurațiile din **FUNCTIONS**.

Procesul este unul de comparație continuă: la fiecare pas, verificăm unde se află numărătorul (**count_val**) față de pragurile setate de noi și decidem dacă ieșirea trebuie să fie „sus” (1 logic) sau „jos” (0 logic).

3.5.2 Logica combinațională și modulele de aliniere

Am ales să implementez decizia logică într-un bloc combinațional (**pwm_logic_result**) pentru a avea un răspuns instantaneu la schimbările numărătorului. Am definit trei scenarii distincte de funcționare, bazate pe registrul **FUNCTIONS**:

- **Modul nealiniat (Unaligned):** Activ dacă **functions[1]** este 1. Aici, semnalul PWM este 1 doar în intervalul cuprins între cele două praguri (**compare1** și **compare2**). Am gândit această implementare special pentru cazurile unde **compare1** < **compare2**, permițând generarea unor impulsuri în orice punct al perioadei.
- **Modul aliniat la dreapta (Align Right):** Dacă **functions[0]** este 1, semnalul începe pe 0 și trece pe 1 doar atunci când numărătorul egalează sau depășește **compare1**.
- **Modul aliniat la stânga (Align Left):** Este modul implicit (default). Semnalul începe pe 1 și trece pe 0 când se atinge pragul **compare1**. Am adăugat o verificare suplimentară pentru a mă asigura că dacă **compare1** este zero, ieșirea rămâne pe 0, evitând astfel impulsuri nedorite.

3.5.3 Mecanismul de siguranță și memorarea stării

O problemă tehnică pe care am anticipat-o a fost comportamentul sistemului atunci când generatorul este dezactivat (**pwm_en** = 0). Fără o gestionare atentă, ieșirea ar putea fluctua sau ar putea rămâne blocată într-o stare care să afecteze dispozitivul controlat.

Pentru a preveni acest lucru, am introdus în logica secvențială un registru numit **pwm_last_state**. Rolul acestuia este simplu, dar critic:

- Atât timp cât `pwm_en` este activ, registrul memorează continuu valoarea calculată de logica PWM.
- În secunda în care PWM-ul este oprit, registrul „îngheață” ultima valoare validă.

Astfel, linia de ieșire nu va sări brusc într-o stare necontrolată, ci va rămâne blocată pe ultimul nivel logic până la o nouă reactivare a perifericului.

3.5.4 Ieșirea finală și sincronizarea

Ultimul pas a fost combinarea acestor elemente printr-un selector (multiplexor) final:

```
assign pwm_out = pwm_en ? pwm_logic_result : pwm_last_state;
```

Această linie de cod asigură că, în regim normal de funcționare, semnalul PWM generat ajunge direct la ieșire. Totuși, în momentul în care `pwm_en` devine inactiv, sistemul face automat trecerea către starea memorată în siguranță, oferind un control robust și predictibil asupra perifericului extern.

4 Concluzii

Proiectul de față a vizat proiectarea și implementarea unui periferic digital complex pentru generarea semnalelor PWM, integrând principii fundamentale de arhitectură a calculatoarelor și design hardware. Prin parcurgerea etapelor de dezvoltare, de la interfațarea SPI până la logica de generare a formelor de undă, s-au obținut următoarele rezultate principale:

- **Modularitate și robustețe:** Arhitectura ierarhică a permis izolarea funcționalităților în blocuri dedicate (Bridge SPI, Decodor, Bloc de Registri, Counter și Generator PWM), facilitând testarea și depanarea sistemului.
- **Control flexibil:** Utilizarea unui set extins de registre de configurare permite ajustarea dinamică a perioadei, a factorului de umplere și a modurilor de aliniere (stânga, dreapta sau nealiniat), oferind o versatilitate similară perifericelor regăsite în microcontrolerele comerciale.
- **Sincronizare și siguranță:** Separarea riguroasă între logica combinațională și cea secvențială a eliminat riscul apariției semnalelor parazite (glitches). Implementarea mecanismului de *latching* pentru starea PWM asigură un comportament predictibil al ieșirii chiar și în momentul dezactivării modulului, protejând astfel sarcinile externe.
- **Scalabilitate:** Integrarea unui prescaler programabil extinde plaja de frecvențe operaționale, făcând acest periferic capabil să deservească o gamă largă de aplicații, de la controlul fin al intensității luminoase până la gestionarea turației motoarelor electrice.

În concluzie, implementarea realizată în Verilog demonstrează un echilibru eficient între utilizarea resurselor hardware și complexitatea funcțională. Proiectul constituie o bază solidă pentru sisteme mai complexe de control, evidențiind importanța unei baze de timp stabile și a unei interfețe de comunicare sigure în designul circuitelor digitale moderne.